

Day 2 — Hands-On Lab Guide

Snowflake for Data Engineers · Fidelity FUIDP · ~6.5 hours

Same model as Day 1 v2: 📄 **Business Problem** → 🛠️ **Guided steps** → 🔬 **Experiments (predict → run → explain)** → ✅ **Checkpoint**. Timebox the guided + experiment parts. **Sync points** at Labs 11 and 14.

Honest time map (390 min)

Lab	Topic	Time
0	Reset & catch-up	15m
8	Streams (CDC)	40m
9	Tasks — Path A pipeline	45m
10	Dynamic Tables — Path B + comparison	40m
11	Airflow orchestration	75m
12	Advanced SQL	45m
13	Query tuning	45m
14	Clustering / SearchOpt / MVs	50m
15	Caching & cost governance	35m

Lab 0 — Reset & Catch-up [15 min]

Run `Day2_Reset_Catchup.sql` top to bottom (it's idempotent — safe even if your Day 1 state is perfect). The final verification block must show **PASS on all 7 rows**, including `trades_large = 5,000,000`.

Instructor: nobody proceeds until their verification block is all-PASS. This is the cheapest 15 minutes of insurance in the course. The script is also your mid-day recovery tool.

✅ **Checkpoint:** 7/7 PASS.

Lab 8 — Streams: Change Data Capture [40 min]

 **Business Problem:** *trades_stream* receives ~50k new rows/day via Snowpipe. Positions must refresh hourly. Reprocessing all 50M historical trades every hour burns a Large warehouse 20 min/hour — ~\$400/day for a 30-second problem. Process **ONLY** what's new.

Guided steps

```
USE SCHEMA FUIDP.RAW; USE WAREHOUSE WH_INGEST;
```

```
-- The bookmark:
```

```
CREATE OR REPLACE STREAM strm_trades ON TABLE trades_stream;
```

```
-- Nothing has changed since creation:
```

```
SELECT COUNT(*) FROM strm_trades;          -- 0
```

```
-- Simulate new trades arriving (stand-in for Snowpipe):
```

```
INSERT INTO trades_stream
```

```
SELECT 'TRD' || LPAD(99000+SEQ4(),8,'0'), 'ACC' || LPAD(UNIFORM(1,200,RANDOM()),5,'0'),  
       'AAPL', 'BUY', UNIFORM(1,500,RANDOM()),195.50,
```

```
       CURRENT_TIMESTAMP()::timestamp_ntz, CURRENT_DATE()+2, 'NASDAQ'  
FROM TABLE(GENERATOR(ROWCOUNT=>500));
```

```
-- The stream sees exactly the delta:
```

```
SELECT COUNT(*) FROM strm_trades;          -- 500
```

```
SELECT METADATA$action, METADATA$isupdate, trade_id FROM strm_trades LIMIT 5;
```

Experiments — predict first

- **(a) Does SELECT consume?** *Predict:* after the SELECTs above, is the stream empty?

```
SELECT COUNT(*) FROM strm_trades;          -- still 500. Peeking ≠ consuming.
```

- **(b) DML consumes.** *Predict:* what happens to the stream after this insert-into-audit?

```
CREATE OR REPLACE TRANSIENT TABLE trades_audit LIKE trades_stream;
```

```
INSERT INTO trades_audit SELECT * EXCLUDE (METADATA$action, METADATA$isupdate, METADATA$RO  
SELECT COUNT(*) FROM strm_trades;          -- 0. Offset advanced: consumed-in-DML.
```



- **(c) Updates show as delete+insert.** Update one row in *trades_stream*, then inspect the stream — you'll see **two** rows (DELETE + INSERT, METADATA\$isupdate=TRUE). Discuss why CDC represents updates this way.

 **Checkpoint:** You can state when an offset moves and demo all three metadata columns.

Lab 9 — Tasks: the Streams+Tasks pipeline (Path A)

[45 min]

 **Business Problem:** *The stream knows WHAT changed; someone must still run the MERGE hourly. The old cron on an ex-employee's VM silently died for 6 days — an advisor quoted week-old valuations. We need scheduling that lives where the data lives, skips runs when there's nothing to do, and leaves an audit trail.*

 **Guided steps**

```
USE SCHEMA FUIDP.ANALYTICS;
```

```
-- Target: current positions per account+symbol
```

```
CREATE OR REPLACE TABLE positions (  
  account_id STRING, symbol STRING, net_quantity NUMBER,  
  last_trade_ts TIMESTAMP_NTZ, updated_at TIMESTAMP_NTZ);
```

```
-- Fresh stream for the pipeline:
```

```
CREATE OR REPLACE STREAM FUIDP.RAW.strm_trades_pipeline ON TABLE FUIDP.RAW.trades_stream;
```

```
-- Root task: every minute (training cadence), ONLY if the stream has data
```

```
CREATE OR REPLACE TASK FUIDP.RAW.task_update_positions  
  WAREHOUSE = WH_INGEST  
  SCHEDULE = '1 MINUTE'  
  WHEN SYSTEM$STREAM_HAS_DATA('FUIDP.RAW.strm_trades_pipeline')
```

```
AS
```

```
  MERGE INTO FUIDP.ANALYTICS.positions p  
  USING (  
    SELECT account_id, symbol,  
           SUM(IFF(side='BUY', quantity, -quantity)) AS delta_qty,  
           MAX(trade_ts) AS max_ts  
    FROM FUIDP.RAW.strm_trades_pipeline  
    GROUP BY account_id, symbol  
  ) s  
  ON p.account_id = s.account_id AND p.symbol = s.symbol  
  WHEN MATCHED THEN UPDATE SET  
    net_quantity = p.net_quantity + s.delta_qty,  
    last_trade_ts = GREATEST(p.last_trade_ts, s.max_ts),  
    updated_at = CURRENT_TIMESTAMP()  
  WHEN NOT MATCHED THEN INSERT  
    (account_id, symbol, net_quantity, last_trade_ts, updated_at)  
    VALUES (s.account_id, s.symbol, s.delta_qty, s.max_ts, CURRENT_TIMESTAMP());
```

```
-- Child task: refresh a valuation summary AFTER positions update
```

```
CREATE OR REPLACE TASK FUIDP.RAW.task_refresh_valuation
```

```
  WAREHOUSE = WH_INGEST  
  AFTER FUIDP.RAW.task_update_positions
```

```
AS
```

```
  CREATE OR REPLACE TABLE FUIDP.ANALYTICS.portfolio_valuation AS  
  SELECT p.account_id, SUM(p.net_quantity * m.price) AS market_value  
  FROM FUIDP.ANALYTICS.positions p  
  JOIN (SELECT symbol, MAX_BY(price, event_ts) AS price  
        FROM FUIDP.STAGING.market_ticks GROUP BY symbol) m  
    ON p.symbol = m.symbol  
  GROUP BY p.account_id;
```

```
-- Tasks are born SUSPENDED. Resume children first, then root:
```

```
ALTER TASK FUIDP.RAW.task_refresh_valuation RESUME;
```

```
ALTER TASK FUIDP.RAW.task_update_positions RESUME;
```

Feed it and watch:

```
INSERT INTO FUIDP.RAW.trades_stream
SELECT 'TRD' || LPAD(98000+SEQ4(),8,'0'), 'ACC' || LPAD(UNIFORM(1,200,RANDOM()),5,'0'),
       ARRAY_CONSTRUCT('AAPL','MSFT','NVDA')[UNIFORM(0,2,RANDOM())]::string,
       IFF(UNIFORM(0,1,RANDOM())=0,'BUY','SELL'),UNIFORM(1,500,RANDOM()),
       ROUND(UNIFORM(100,500,RANDOM()),2),CURRENT_TIMESTAMP()::timestamp_ntz,
       CURRENT_DATE()+2,'NYSE'
FROM TABLE(GENERATOR(ROWCOUNT=>300));
```

```
-- Within ~60-90s:
```

```
SELECT COUNT(*), MAX(updated_at) FROM FUIDP.ANALYTICS.positions;
```

```
SELECT * FROM TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START=>DATEADD('minute',-15,CURRENT_TIMESTAMP())))
ORDER BY SCHEDULED_TIME DESC;
```

Experiments — predict first

- **(a) The no-data skip.** With the stream empty, *predict* what `TASK_HISTORY` shows for the next minute's run. (State = `SKIPPED`, condition false — **zero compute**. This is the \$400/day fix.)
- **(b) Exactly-once.** Insert 100 rows, let the task fire, then check the stream and positions. *Predict*: if the task fired twice in a row, would rows double-count? Why not? (The `MERGE` consumed the stream in the same transaction — offset moved.)

 **Before leaving this lab:** `ALTER TASK ... SUSPEND;` both tasks (we re-orchestrate in Lab 11 — and idle 1-minute tasks burn credits all day).

 **Checkpoint:** Pipeline runs hands-free; you've seen `SKIPPED` (no-cost) runs and suspended everything.

Lab 10 — Dynamic Tables: the same pipeline, declaratively (Path B) [40 min]

 **Business Problem:** *Path A* took a stream + a task + a `MERGE` + schedule wiring — ~80 lines of plumbing for ONE derived table. Fidelity's analytics layer needs ~40 of these. That's 40 choreographies to maintain and onboard people into. Can we declare WHAT each table contains and make freshness Snowflake's problem?

Guided steps

```
USE SCHEMA FUIDP.ANALYTICS;
```

```
-- The ENTIRE positions pipeline, declaratively:
```

```
CREATE OR REPLACE DYNAMIC TABLE dt_positions
  TARGET_LAG = '1 minute'      -- training cadence; prod would be 5-15 min
  WAREHOUSE  = WH_INGEST
AS
SELECT account_id, symbol,
       SUM(IFF(side='BUY', quantity, -quantity)) AS net_quantity,
       MAX(trade_ts) AS last_trade_ts
FROM FUIDP.RAW.trades_stream
GROUP BY account_id, symbol;
```

```
-- Chain: valuation built ON the dynamic table (automatic dependency)
```

```
CREATE OR REPLACE DYNAMIC TABLE dt_portfolio_valuation
  TARGET_LAG = DOWNSTREAM      -- refresh when my consumers need me
  WAREHOUSE  = WH_INGEST
AS
SELECT p.account_id, SUM(p.net_quantity * m.price) AS market_value
FROM dt_positions p
JOIN (SELECT symbol, MAX_BY(price, event_ts) AS price
      FROM FUIDP.STAGING.market_ticks GROUP BY symbol) m
  ON p.symbol = m.symbol
GROUP BY p.account_id;
```

Feed the same kind of insert as Lab 9 into `trades_stream`, then watch:

```
SELECT * FROM TABLE(INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY())
ORDER BY REFRESH_START_TIME DESC LIMIT 10;
-- Snowsight: Data » FUIDP » ANALYTICS » dt_positions » Refresh History (visual DAG!)
SELECT COUNT(*) FROM dt_positions;
```

Experiments — predict first

- **(a) Incremental or full?** In `DYNAMIC_TABLE_REFRESH_HISTORY`, check the refresh action for `dt_positions`. *Predict first*. (Aggregations like this are typically **incremental** — Snowflake wrote the delta logic you hand-wrote in Lab 9.)
- **(b) No new data.** *Predict:* with no inserts for 2 minutes, what does refresh history show? (Skips/no-op refreshes — the `SYSTEM$STREAM_HAS_DATA` behavior, for free.)
- **(c) Lag truth.** Compare `TARGET_LAG` vs actual lag in the refresh history columns. Is the promise being met?

⚠️ **Before leaving:** suspend both dynamic tables (1-minute lag polls all day otherwise).

✅ **Checkpoint:** Both pipelines produce matching position counts.

Lab 11 — Apache Airflow orchestration [75 min] · SYNC POINT

📋 **Business Problem:** *Fidelity's daily batch isn't only Snowflake: confirm the custodian file landed, transform, run data-quality gates, refresh analytics, notify BI on Slack, page on failure. Half of that isn't SQL — native tasks can't poll SFTP or post to Slack. And Compliance wants ONE pane of glass for every overnight job. We need a general-purpose orchestrator driving the Snowflake bricks we just built.*

Uses the `airflow_bundle/` provided: `docker-compose.yaml`, `requirements.txt`, and `dags/fuidp_pipeline_dag.py` (a skeleton you'll complete). Keep Day 1's Docker running knowledge handy.

🔧 Guided steps

11a. Bring up Airflow

```
cd airflow_bundle
docker compose up -d          # postgres + webserver + scheduler (LocalExecutor)
docker compose ps             # all healthy?
# UI: http://localhost:8080   (user: airflow / pass: airflow)
```

11b. Create the Snowflake connection — UI » **Admin » Connections » +**:

- Conn Id: `snowflake_fuidp` · Conn Type: **Snowflake**
- Login/Password: your lab user/password · Account: `<account_locator>` · Warehouse: `WH_INGEST`
· Database: `FUIDP` · Schema: `RAW` · Role: `ACCOUNTADMIN` (lab only — discuss why prod uses a scoped service role)
- **Test** the connection before saving.

11c. Read the skeleton DAG (`dags/fuidp_pipeline_dag.py`). It defines the shape; three tasks are marked `# TODO` . The flow:

```
check_freshness → transform_positions → dq_gate → refresh_analytics → notify
```

11d. Complete the *TODOs*. Each `TODO` is a `SQL_*` block near the top of the skeleton holding a `SELECT 'replace me'` placeholder. Replace each with the SQL below — these are self-contained (no

need to flip back to Lab 9).

TODO #1 — SQL_TRANSFORM_POSITIONS : recompute current net positions from `trades_stream` and UPSERT them. This is a *full recompute*, so it's idempotent — re-running writes identical numbers, never double-counts. Pass it as a **Python list of two statements** so both run in one task:

```
SQL_TRANSFORM_POSITIONS = [
    """
    CREATE TABLE IF NOT EXISTS FUIDP.ANALYTICS.positions (
        account_id    STRING,
        symbol         STRING,
        net_quantity   NUMBER,
        last_trade_ts  TIMESTAMP_NTZ,
        updated_at     TIMESTAMP_NTZ
    )
    """,
    """
    MERGE INTO FUIDP.ANALYTICS.positions p
    USING (
        SELECT account_id, symbol,
               SUM(IFF(side='BUY', quantity, -quantity)) AS net_quantity,
               MAX(trade_ts) AS last_trade_ts
        FROM FUIDP.RAW.trades_stream
        GROUP BY account_id, symbol
    ) s
    ON p.account_id = s.account_id AND p.symbol = s.symbol
    WHEN MATCHED THEN UPDATE SET
        net_quantity   = s.net_quantity,
        last_trade_ts  = s.last_trade_ts,
        updated_at     = CURRENT_TIMESTAMP()
    WHEN NOT MATCHED THEN INSERT
        (account_id, symbol, net_quantity, last_trade_ts, updated_at)
        VALUES (s.account_id, s.symbol, s.net_quantity, s.last_trade_ts, CURRENT_TIMESTAMP())
    """,
]
```

TODO #2 — SQL_DQ_GATE : three checks; each **fails the task** the moment its invariant breaks. The `1/0` forces a divide-by-zero error (same trick as `check_freshness`). A **list** runs them in order; the first failing one aborts the task:


```
SQL_DQ_GATE = [
    # check 1: positions not empty
    "SELECT IFF(COUNT(*) > 0, 1, 1/0) AS dq_positions_not_empty "
    "FROM FUIDP.ANALYTICS.positions",
    # check 2: no NULL account_id
    "SELECT IFF(COUNT(*) = 0, 1, 1/0) AS dq_no_null_account "
    "FROM FUIDP.ANALYTICS.positions WHERE account_id IS NULL",
    # check 3: no future-dated trades
    "SELECT IFF(COUNT(*) = 0, 1, 1/0) AS dq_no_future_trades "
    "FROM FUIDP.ANALYTICS.positions WHERE last_trade_ts > CURRENT_TIMESTAMP()",
]
```

TODO #3 — SQL_REFRESH_ANALYTICS : rebuild portfolio_valuation from the fresh positions joined to the latest price per symbol:

```
SQL_REFRESH_ANALYTICS = """
CREATE OR REPLACE TABLE FUIDP.ANALYTICS.portfolio_valuation AS
SELECT p.account_id,
       SUM(p.net_quantity * m.price) AS market_value
FROM FUIDP.ANALYTICS.positions p
JOIN (
    SELECT symbol, MAX_BY(price, event_ts) AS price
    FROM FUIDP.STAGING.market_ticks
    GROUP BY symbol
) m ON p.symbol = m.symbol
GROUP BY p.account_id;
"""
```

The skeleton's transform_positions and dq_gate operators already pass these SQL_* names to sql=, and SQLExecuteQueryOperator accepts a list as readily as a string — no operator changes needed, just swap the placeholder values.

11e. Run it — unpause fuidp_pipeline in the UI, **Trigger DAG**, watch the Graph view go green, click a task » **Logs** to see the SQL and row counts.

Experiments — predict first

- **(a) Retry semantics.** The skeleton sets retries=2, retry_delay=30s . *Predict:* if transform_positions fails transiently once, what do the logs and the Gantt view show? Force it: suspend WH_INGEST resize mid-run or briefly break the conn, watch a retry succeed.
- **(b) Idempotency.** Trigger the DAG **twice in a row** with no new data. *Predict:* do positions double? (No — transform_positions recomputes each account/symbol's absolute net position and UPDATES in place, so a second run with the same data writes identical numbers. This is why we built with MERGE.)

✅ **Checkpoint (SYNC):** DAG green end-to-end; the DQ gate is wired into the flow.

Lab 12 — Advanced SQL: windows, MERGE, procedures [45 min]

 **Business Problem:** *Portfolio Analytics has three asks the current reports can't do: (1) running cost basis per account as trades accumulate; (2) duplicate trade messages from the venue feed — keep exactly one, no delete-and-reload; (3) daily P&L that UPDATES existing rows when late trades arrive and INSERTs new ones — atomically. Each is a babysat multi-step script today.*

Guided steps

(1) *Running cost basis — window functions:*

```
USE SCHEMA FUIDP.ANALYTICS; USE WAREHOUSE WH_ANALYTICS;
SELECT account_id, symbol, trade_ts, side, quantity, price,
       SUM(IFF(side='BUY', quantity*price, -quantity*price))
       OVER (PARTITION BY account_id, symbol ORDER BY trade_ts
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_cost_basis
FROM FUIDP.RAW.trades
WHERE account_id = 'ACC00042'
ORDER BY symbol, trade_ts;
```

(2) *Dedup — QUALIFY:*

```
-- Manufacture duplicates first:
INSERT INTO FUIDP.RAW.trades SELECT * FROM FUIDP.RAW.trades LIMIT 50;
SELECT COUNT(*), COUNT(DISTINCT trade_id) FROM FUIDP.RAW.trades;    -- gap = 50 dups

-- One clause, no temp tables:
CREATE OR REPLACE TABLE FUIDP.STAGING.trades_dedup AS
SELECT * FROM FUIDP.RAW.trades
QUALIFY ROW_NUMBER() OVER (PARTITION BY trade_id ORDER BY trade_ts DESC) = 1;
SELECT COUNT(*), COUNT(DISTINCT trade_id) FROM FUIDP.STAGING.trades_dedup;    -- equal
```

(3) *Daily P&L — MERGE wrapped in a procedure:*

```

CREATE OR REPLACE TABLE daily_pnl (
  account_id STRING, trade_date DATE, realized_flow NUMBER(18,2),
  trade_count NUMBER, updated_at TIMESTAMP_NTZ);

CREATE OR REPLACE PROCEDURE compute_daily_pnl(p_date DATE)
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
  rows_merged INTEGER;
BEGIN
  MERGE INTO daily_pnl t
  USING (
    SELECT account_id, trade_ts::date AS trade_date,
           SUM(IFF(side='SELL', quantity*price, -quantity*price)) AS realized_flow,
           COUNT(*) AS trade_count
    FROM FUIDP.STAGING.trades_dedup
    WHERE trade_ts::date = :p_date
    GROUP BY 1,2
  ) s
  ON t.account_id = s.account_id AND t.trade_date = s.trade_date
  WHEN MATCHED THEN UPDATE SET realized_flow = s.realized_flow,
    trade_count = s.trade_count, updated_at = CURRENT_TIMESTAMP()
  WHEN NOT MATCHED THEN INSERT VALUES
    (s.account_id, s.trade_date, s.realized_flow, s.trade_count, CURRENT_TIMESTAMP());
  rows_merged := SQLROWCOUNT;
  RETURN 'P&L merged for ' || :p_date || ' - rows affected: ' || rows_merged;
END;
$$;

CALL compute_daily_pnl('2024-06-03');
SELECT * FROM daily_pnl ORDER BY realized_flow DESC LIMIT 10;

```

Experiments — predict first

- **(a) Idempotency proof.** *Predict:* CALL compute_daily_pnl('2024-06-03') again — duplicate rows or same counts? Why? (WHEN MATCHED updates in place — the same property Lab 11's DAG retries rely on.)
- **(b) Late trade.** Insert one extra 2024-06-03 trade into trades_dedup , re-CALL, and verify exactly one account's row changed (updated_at moved). This is the late-arrival scenario from the problem statement.

✅ **Checkpoint:** Three business asks, three single-statement answers; the procedure is idempotent and becomes the transform brick both orchestrators call.

Lab 13 — Query performance tuning [45 min]

 **Business Problem:** "My portfolio query took 11 minutes; it used to take 40 seconds." The reflex fix — XL warehouse — works at 16x cost, and nobody knows if it's a fix or a bandage. Diagnose before you spend.

All on `trades_large` (5M rows). You will *manufacture* each pathology, read its signature in the Query Profile, and apply the **correct** fix.

 **Guided steps +  experiments (interleaved by design)**

Pathology 1 — poor pruning:

```
USE WAREHOUSE WH_ANALYTICS;  
SELECT COUNT(*), AVG(price) FROM FUIDP.RAW.trades_large  
WHERE trade_ts BETWEEN '2024-06-10' AND '2024-06-12';
```

Open the profile. *Predict first:* partitions scanned vs total? (Near-total — the table was generated in random ts order; nothing prunes.) **Correct fix:** organize the data (Lab 14's clustering) — *not* a bigger warehouse. Park it; we fix it next lab and re-measure.

Pathology 2 — spillage: **Spilling** = an operator (here the full sort behind `ROW_NUMBER + ORDER BY` over 5M rows) needs more memory than the warehouse node has. It overflows first to **local** SSD, then — if that fills too — to **remote** object storage (much slower). A bigger warehouse has more nodes and therefore more total RAM, so this is the one pathology where upsizing is the *correct* fix.

Step 1 — force the spill on the smallest warehouse. The `USE_CACHED_RESULT=FALSE` line is essential: without it, your second run returns instantly from the result cache and there is nothing to compare.

```
ALTER WAREHOUSE WH_ANALYTICS SET WAREHOUSE_SIZE='XSMALL';  
ALTER SESSION SET USE_CACHED_RESULT = FALSE; -- so re-runs actually recompute  
SELECT account_id, symbol, trade_ts, price,  
       ROW_NUMBER() OVER (ORDER BY price DESC, trade_ts, trade_id) AS rn  
FROM FUIDP.RAW.trades_large  
ORDER BY price DESC, trade_ts  
LIMIT 100000;
```

Step 2 — read the signature in the Query Profile. After it finishes, open the profile: click the **query ID** above the results (or **Monitoring » Query History** → click this query) → **Query Profile** tab.

- Click the most expensive node — the **Sort** operator.
- In the right-hand **Statistics / Profile Overview** panel, find the **Spilling** group:

- **Bytes spilled to local storage** — expect a large value (hundreds of MB+).
- **Bytes spilled to remote storage** — on XSMALL you may see this too; it's the painful one.
- Note the **Total Execution Time** (or the **Duration** in the results pane) — this is your before-baseline.

Predict: at XSMALL, both spill metrics are > 0 and elapsed time is high.

Step 3 — apply the fix (more memory) and re-measure. Resize up and rerun the **identical** query:

```
ALTER WAREHOUSE WH_ANALYTICS SET WAREHOUSE_SIZE='MEDIUM';
SELECT account_id, symbol, trade_ts, price,
       ROW_NUMBER() OVER (ORDER BY price DESC, trade_ts, trade_id) AS rn
FROM FUIDP.RAW.trades_large
ORDER BY price DESC, trade_ts
LIMIT 100000;
```

Open the new profile, click the Sort node, expand **Spilling** again. *Expect:* local spill **much smaller or zero**, remote spill **gone**, and Total Execution Time noticeably lower — MEDIUM has ~4× the memory of XSMALL (4 nodes vs 1), so the sort now fits in RAM. **This** is the legitimate use of a bigger warehouse: the work is a genuine large sort, not fixable by reorganizing data or rewriting SQL (contrast Pathology 1, where upsizing would burn money without fixing the root cause).

Step 4 — reset:

```
ALTER WAREHOUSE WH_ANALYTICS SET WAREHOUSE_SIZE='SMALL';
ALTER SESSION UNSET USE_CACHED_RESULT;  -- restore default caching
```

Pathology 3 — join explosion: A **join explosion** is when the join's ON condition isn't a true key match, so each input row pairs with many rows on the other side and the join *emits far more rows than it consumes*. Here the condition references only `a.advisor_id` against a constant — no link to `t` at all — so it behaves like a near-Cartesian product. No warehouse size fixes this; the row count itself is the problem.

Step 1 — run the broken join. It can produce billions of rows — **don't wait for it to finish**; open the profile while it runs and cancel once you've seen the signature.

```
-- Bad join: condition doesn't tie t to a on a real key.
SELECT t.trade_id, a.advisor_id
FROM FUIDP.RAW.trades_large t
JOIN FUIDP.RAW.accounts a ON a.advisor_id = 'ADV' || LPAD(UNIFORM(1,20,RANDOM()),3,'0');
```

Step 2 — read the signature in the Query Profile. Open the profile (it works on a **running** query): click the query ID → **Query Profile** tab.

- Find the **Join** node — it'll be the dominant, most expensive operator.
- Look at the **row counts on the arrows** in and out of it: input edges carry ~5M (trades) and ~a few thousand (accounts); the **output edge carries orders of magnitude more** — that gap *is* the explosion.
- In the node's **Statistics**, watch the output row count climbing while it runs. That, plus a join node dwarfing everything else, is the fingerprint.

Predict: output rows >> the larger of the two inputs (not ≈ 5M).

Step 3 — cancel and apply the correct fix. Cancel the runaway query (the **X / Cancel** in the worksheet, or `SELECT SYSTEM$CANCEL_ALL_QUERIES();` for your session). Then join on the real key so it's 1:1 per trade:

```
SELECT t.trade_id, a.advisor_id
FROM FUIDP.RAW.trades_large t
JOIN FUIDP.RAW.accounts a ON a.account_id = t.account_id;  -- sane: 1 account per trade
```

Open this profile and confirm the **join output ≈ number of trades** (~5M), not a multiple of it. **The fix was the SQL, not the hardware** — a bigger warehouse would just produce the same wrong billions of rows faster and more expensively. (No reset needed; this pathology touched no warehouse or session settings.)

✅ **Checkpoint:** You can read all three signatures cold and name the matching fix — including which one justifies more compute and which two don't.

Lab 14 — Clustering, Search Optimization & Materialized Views [50 min] · SYNC POINT

 **Business Problem:** *Three teams, three slow patterns, ONE 5M-row table (5B in prod): Risk scans date ranges; Client Services needs sub-second account lookups; the exec dashboard re-runs the same sector rollup hundreds of times daily. There's no single fix — there are three priced features, and the wrong assignment wastes money.*

Guided steps

Feature 1 — clustering (for Risk's range scans):

```
-- BEFORE: capture the baseline from Lab 13's pathology-1 profile numbers.
ALTER TABLE FUIDP.RAW.trades_large CLUSTER BY (trade_ts);

-- Reclustering is background; give it a few minutes, monitor:
SELECT SYSTEM$CLUSTERING_INFORMATION('FUIDP.RAW.trades_large','(trade_ts)');

-- AFTER (run once average_depth has dropped):
SELECT COUNT(*), AVG(price) FROM FUIDP.RAW.trades_large
WHERE trade_ts BETWEEN '2024-06-10' AND '2024-06-12';

-- Profile: partitions scanned should now be a small fraction of total.
```

Feature 2 — search optimization (for Client Services' point lookups):

```
-- BEFORE:
SELECT * FROM FUIDP.RAW.trades_large WHERE account_id='ACC00042'; -- note time + partitions
ALTER TABLE FUIDP.RAW.trades_large ADD SEARCH OPTIMIZATION ON EQUALITY(account_id);
SHOW TABLES LIKE 'trades_large'; -- search_optimization=ON; wait for active build
-- AFTER: rerun the lookup; compare partitions scanned + elapsed.
```

Feature 3 — materialized view (for the dashboard's repeated rollout):

```
CREATE OR REPLACE MATERIALIZED VIEW FUIDP.ANALYTICS.mv_symbol_summary AS
SELECT symbol, COUNT(*) AS trade_count, SUM(quantity) AS total_qty, AVG(price) AS avg_price
FROM FUIDP.RAW.trades_large GROUP BY symbol;

SELECT * FROM FUIDP.ANALYTICS.mv_symbol_summary; -- instant
-- vs the same aggregate on the base table: compare profiles.
```

Experiments — predict first

- **(a) MV freshness.** Insert 1,000 rows into `trades_large`. *Predict:* is the MV stale? Query it — Snowflake serves correct results (blending the delta) while maintenance catches up. Discuss the hidden cost: every base-table change feeds MV maintenance.
- **(b) SearchOpt scope.** *Predict:* does the search-optimized lookup help
WHERE trade_ts BETWEEN ... ? (No — we built ON EQUALITY(account_id) ; ranges are clustering's job. Features are scoped, not magic.)

✅ **Checkpoint (SYNC):** Before/after numbers captured for all three features.

Lab 15 — Caching & cost governance [35 min]

 **Business Problem:** *The Snowflake bill doubled in a quarter and nobody can say which team, workload, or query did it. An intern's runaway cross-join burned 200 credits overnight with no guardrail. Finance wants attribution; the platform team needs circuit breakers — or loses budget credibility.*

Guided steps

(1) Caches — the free wins:

```
-- Result cache: run any Lab 14 aggregate twice, identical text.
SELECT symbol, COUNT(*), AVG(price) FROM FUIDP.RAW.trades_large GROUP BY symbol;
-- Rerun verbatim: ~ms, 0 partitions. Profile says RESULT REUSE.
-- Now defeat it the way dashboards accidentally do:
SELECT symbol, COUNT(*), AVG(price), CURRENT_TIMESTAMP() FROM FUIDP.RAW.trades_large GROUP BY :
-- Full recompute. One needless non-deterministic column = cache gone.
```



(2) The circuit breaker — resource monitor:

```
CREATE OR REPLACE RESOURCE MONITOR rm_fuidp_training
  CREDIT_QUOTA = 5  FREQUENCY = DAILY  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 75 PERCENT DO NOTIFY
           ON 100 PERCENT DO SUSPEND;
ALTER WAREHOUSE WH_ANALYTICS SET RESOURCE_MONITOR = rm_fuidp_training;
SHOW RESOURCE MONITORS;
```

(3) Attribution — tags + ACCOUNT_USAGE:

```
ALTER SESSION SET QUERY_TAG = 'team:risk;pipeline:fuidp;lab:15';
SELECT COUNT(*) FROM FUIDP.RAW.trades_large WHERE symbol='NVDA';
ALTER SESSION UNSET QUERY_TAG;

-- The chargeback query (note: ACCOUNT_USAGE can lag ~45 min):
SELECT QUERY_TAG, COUNT(*) AS queries,
       SUM(TOTAL_ELAPSED_TIME)/1000 AS total_seconds,
       SUM(BYTES_SCANNED)/POWER(1024,3) AS gb_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME > DATEADD('day',-1,CURRENT_TIMESTAMP()) AND QUERY_TAG <> ''
GROUP BY QUERY_TAG ORDER BY total_seconds DESC;
```




Experiment — find today's most expensive query. *Predict first which lab produced it (most bets: Lab 13's spillage sort or Lab 0's 5M-row generate), then:*

```
SELECT QUERY_TEXT, WAREHOUSE_NAME, TOTAL_ELAPSED_TIME/1000 AS sec,
       PARTITIONS_SCANNED, BYTES_SPILLED_TO_LOCAL_STORAGE
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE START_TIME > DATEADD('hour', -8, CURRENT_TIMESTAMP()) AND WAREHOUSE_NAME IS NOT NULL
ORDER BY TOTAL_ELAPSED_TIME DESC LIMIT 10;
-- (If ACCOUNT_USAGE hasn't caught up, use INFORMATION_SCHEMA.QUERY_HISTORY() – last 7 days, no
```



Checkpoint: Monitor attached and visible; a tagged query attributable in history; the day's most expensive query identified and explained.

Wrap-up & teardown [5 min]

The raw firehose now becomes governed analytics: CDC (streams) → the pipeline twice (tasks vs dynamic tables) → one Airflow control plane with a DQ gate → diagnosed-not-guessed performance → three priced features correctly triaged → attribution + circuit breakers.

```
-- Suspend everything that polls or schedules:
ALTER TASK FUIDP.RAW.task_update_positions SUSPEND;
ALTER TASK FUIDP.RAW.task_refresh_valuation SUSPEND;
ALTER DYNAMIC TABLE FUIDP.ANALYTICS.dt_positions SUSPEND;
ALTER DYNAMIC TABLE FUIDP.ANALYTICS.dt_portfolio_valuation SUSPEND;
ALTER WAREHOUSE WH_INGEST SUSPEND; ALTER WAREHOUSE WH_ANALYTICS SUSPEND;
```

```
docker compose down      # Airflow off; volumes persist for Day 3 reuse
```

Keep all FUIDP objects — Day 3 (Time Travel, Snowpark, sharing, governance) starts here, and the PII in `accounts` is about to get masked.